МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Національний аерокосмічний університет
«Харківський авіаційний інститут»

Факультет систем управління літальних апаратів
Кафедра систем управління літальних апаратів

Лабораторна робота № 8
з дисципліни «Алгоритмізація та програмування»
на тему «"Реалізація алгоритмів сортування та робота з файлами на мові С ++"»

ХАІ.301.G7.319.30 ЛР

Виконав студент гр._____319гр._____

16.03.2026 Руденко Валентин
(підпис, дата) (П.І.Б)

Перевірив
_____ к.т.н., доц. Олена ГАВРИЛЕНКО
(підпис, дата) (П.І.Б.)

2026

МЕТА РОБОТИ

Вивчити теоретичний матеріал по алгоритмам обробки масивів на мові C++, а також бібліотеки для роботи з файлами і реалізувати оголошення, введення з файлу, обробку і виведення в файл одновимірних і двовимірних масивів на мові C ++ в середовищі Visual Studio.

ПОСТАНОВКА ЗАДАЧІ

Завдання 1. За допомогою текстового редактору створити текстовий файл «array_in_*n*.txt» з елементами вихідного масиву (n - номер варіанта). У програмі на C++ зчитати і перетворити цей масив відповідно до свого варіанту завдання (див. лаб.роб.№6, завд.2), ім'я файлу і необхідні змінні ввести з консолі. Вивести результати у файл «array_out_*n*.txt».

Завдання 2. За допомогою текстового редактору створити текстовий файл «matr_in_*n*.txt» з елементами вихідного двовимірного масиву (*n - номер варіанта*). У програмі зчитати і обробити матрицю відповідно до свого варіанту завдання (лаб.роб.№7, завд.1), ім'я файлу і необхідні змінні ввести з консолі. Дописати результати в той же файл.

Завдання 3. Вивчити метод сортування відповідно до свого варіанту (див. табл. 1), проаналізувати його складність і продемонструвати на прикладі з 7-ми елементів (відповідно до свого варіанту). Реалізувати у вигляді окремої функції алгоритм сортування елементів масиву. Зчитування і виведення відсортованого масиву організувати на файлах.

Завдання 4. Введення, виведення, обробку масивів реалізувати окремими функціями з параметрами. Структурувати проєкт програми для виконання завдань 1-3 наступним чином:

main.cpp //основна функція і три функції для 3х завдань

array_utils.h //заголовки функцій для роботи з
//одновимірними масивами (завдання1,3)

array_utils.cpp //визначення функцій для роботи з
//одновимірними масивами (завдання1,3)

matrix_utils.h //заголовки функцій для роботи з
//двовимірними масивами (завдання2)

matrix_utils.cpp //визначення функцій для роботи з
//двовимірними масивами (завдання2)

Завдання 5. Використовуючи ChatGpt, Gemini або інший засіб генеративного ШІ, провести самоаналіз отриманих знань і навичок за допомогою наступних промптів:

1) «Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання <середнього> рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду, що є у файлі звіту, і до теоретичних відомостей, що є у файлі лекції»

2) «Проаналізуй повноту, правильність відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Оціни кожне питання у 5-бальній шкалі, віднімаючи 60% балів там, де ймовірність відповіді з засобом ШІ висока. Обчисли загальну середню оцінку»

ВИКОНАННЯ РОБОТИ

Завдання 1. За допомогою текстового редактору створити текстовий файл «array_in_*n*.txt» з елементами вихідного масиву (n - номер варіанта). У програмі на С++ зчитати і перетворити цей масив відповідно до свого варіанту завдання (див. лаб.роб.№6, завд.2), ім'я файлу і необхідні змінні ввести з консолі. Вивести результати у файл «array_out_*n*.txt».

```
int n;
fin >> n;
vector<double> arr(n);
for (int i = 0; i < n; i++)
   fin >> arr[i];

// Вивід всіх парних елементів у зворотньому порядку
fout << "Even elements in reverse: ";
int countEven = 0;
for (int i = n - 1; i >= 0; i--) {
   if (int(arr[i]) % 2 == 0) {
      fout << arr[i] << " ";
      countEven++;
   }
```

Лістинг коду вирішення задачі наведено в дод. А (стор. 7).
Екрани роботи програми показано на рис. Б.(стор. 10).

Завдання 2. За допомогою текстового редактору створити текстовий файл «matr_in_*n*.txt» з елементами вихідного двовимірного масиву (*n* - *номер варіанта*). У програмі зчитати і обробити матрицю відповідно до свого варіанту завдання (лаб.роб.№7, завд.1), ім'я файлу і необхідні змінні ввести з консолі. Дописати результати в той же файл.

```
int m, n;
fin >> m >> n;
vector<vector<double>> matrix(m, vector<double>(n));

for (int i = 0; i < m; i++)
   for (int j = 0; j < n; j++)
      fin >> matrix[i][j];

// Swap left-top and right-bottom quarters (simplified)
int halfM = m / 2, halfN = n / 2;
for (int i = 0; i < halfM; i++)
   for (int j = 0; j < halfN; j++)
      swap(matrix[i][j], matrix[i + halfM][j + halfN]);

fout << "Matrix after swap:\n";
```

```
for (int i = 0; i < m; i++) {
    for (int j = 0; j < n; j++)
        fout << matrix[i][j] << "\t";
    fout << "\n";
}
```

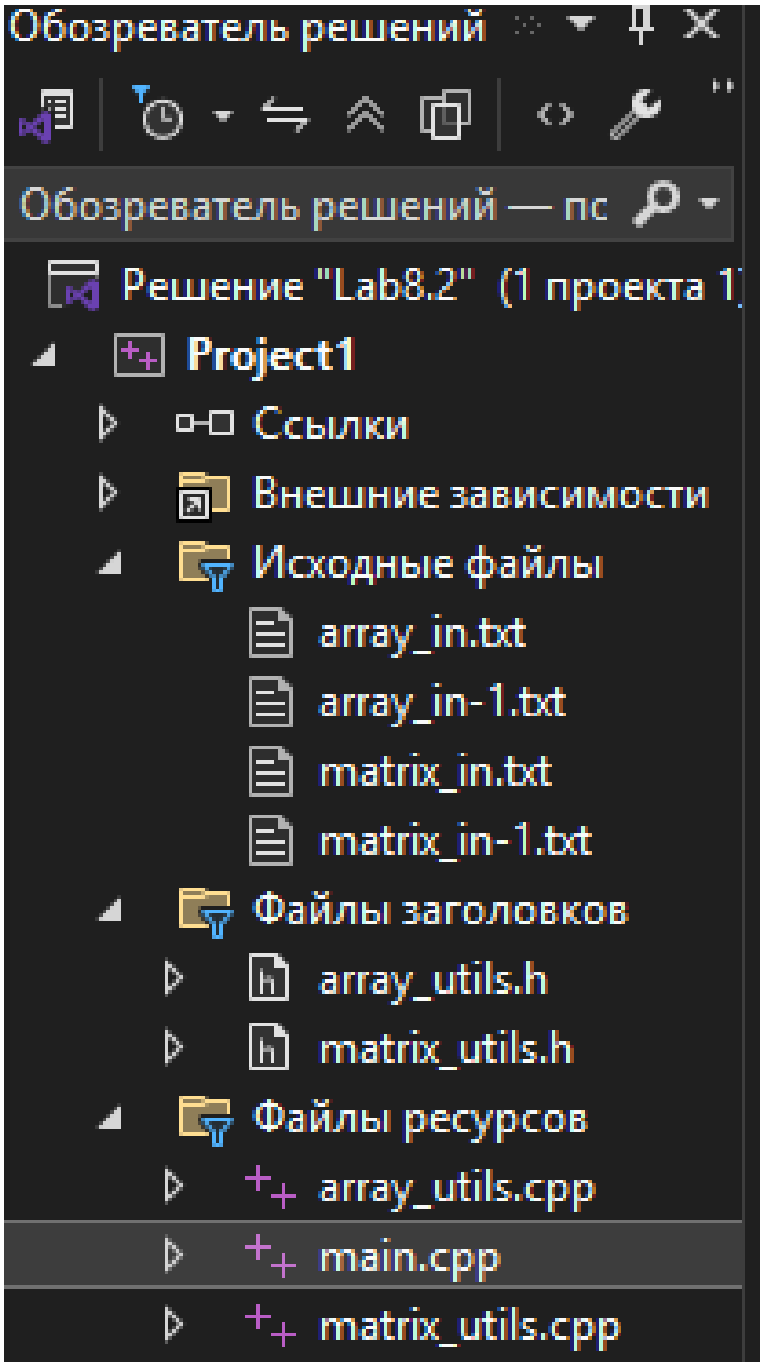
Лістинг коду вирішення задачі наведено в дод. А (стор. 9).
Екрани роботи програми показано на рис. Б.(стор. 11).

Завдання 3. Вивчити метод сортування відповідно до свого варіанту (див. табл. 1), проаналізувати його складність і продемонструвати на прикладі з 7-ми елементів (відповідно до свого варіанту). Реалізувати у вигляді окремої функції алгоритм сортування елементів масиву. Зчитування і виведення відсортованого масиву організувати на файлах.

```
int n;
fin >> n;
vector<double> arr(n);
for (int i = 0; i < n; i++)
    fin >> arr[i];

// Bubble sort
for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - 1 - i; j++) {
        if (arr[j] > arr[j + 1])
            swap(arr[j], arr[j + 1]);
    }
}

fout << "Sorted array: ";
for (int i = 0; i < n; i++)
    fout << arr[i] << " ";
fout << "\n";

cout << "Task 3 completed. Check array_out.txt\n";
```

Лістинг коду вирішення задачі наведено в дод. А (стор. 8).
Екрани роботи програми показано в дод. Б.(стор. 12).
Діаграма активності для алгоритму вирішення задачі в дод. В (стор. 13, 14)
Схема сортування алгоритму «бульбашки» в дод. Г (стор. 15)

Завдання 4. Введення, виведення, обробку масивів реалізувати окремими функціями з параметрами.



ВИСНОВКИ

Було вивчено теоретичний матеріал по алгоритмам обробки масивів на мові C++, а також бібліотеки для роботи з файлами і реалізовано оголошення, введення з файлу, обробку і виведення в файл одновимірних і двовимірних масивів на мові C ++ в середовищі Visual Studio.

ДОДАТОК А
Лістинг коду програми

```
#ifndef ARRAY_UTILS_H
#define ARRAY_UTILS_H

#include <string>

void task1_array(const std::string filename);
void task3_bubble_sort(const std::string filename);

#endif
```

```
#include "array_utils.h"
#include <iostream>
#include <fstream>
#include <vector>

using namespace std;

// Завдання 1: аналіз одновимірного масиву
void task1_array(const string filename) {
    ifstream fin(filename);
    ofstream fout("array_out.txt", ios::app);

    int n;
    fin >> n;
    vector<double> arr(n);
    for (int i = 0; i < n; i++)
        fin >> arr[i];

    // Вивід всіх парних елементів у зворотньому порядку
    fout << "Even elements in reverse: ";
    int countEven = 0;
    for (int i = n - 1; i >= 0; i--) {
        if (int(arr[i]) % 2 == 0) {
            fout << arr[i] << " ";
            countEven++;
        }
    }
    fout << "\nNumber of even elements: " << countEven << "\n";
```

```
    cout << "Task 1 completed. Check array_out.txt\n";

    fin.close();
    fout.close();
}

// Завдання 3: сортування бульбашкою
void task3_bubble_sort(const string filename) {
    ifstream fin(filename);
    ofstream fout("array_out.txt", ios::app);

    int n;
    fin >> n;
    vector<double> arr(n);
    for (int i = 0; i < n; i++)
        fin >> arr[i];

    // Bubble sort
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
            if (arr[j] > arr[j + 1])
                swap(arr[j], arr[j + 1]);
        }
    }

    fout << "Sorted array: ";
    for (int i = 0; i < n; i++)
        fout << arr[i] << " ";
    fout << "\n";

    cout << "Task 3 completed. Check array_out.txt\n";

    fin.close();
    fout.close();
}
```

```
#ifndef MATRIX_UTILS_H
#define MATRIX_UTILS_H

#include <string>

void task2_matrix(const std::string filename);

#endif
```

```
#include "matrix_utils.h"
#include <iostream>
#include <fstream>
#include <vector>

using namespace std;

void task2_matrix(const string filename) {
    ifstream fin(filename);
    ofstream fout("matrix_out.txt", ios::app);

    int m, n;
    fin >> m >> n;
    vector<vector<double>> matrix(m, vector<double>(n));

    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
            fin >> matrix[i][j];

    // Swap left-top and right-bottom quarters (simplified)
    int halfM = m / 2, halfN = n / 2;
    for (int i = 0; i < halfM; i++)
        for (int j = 0; j < halfN; j++)
            swap(matrix[i][j], matrix[i + halfM][j + halfN]);

    fout << "Matrix after swap:\n";
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++)
            fout << matrix[i][j] << "\t";
        fout << "\n";
    }

    cout << "Task 2 completed. Check matrix_out.txt\n";

    fin.close();
    fout.close();
}
```

ДОДАТОК Б
Скрін-шоти вікна виконання програми

```
===== MENU =====
1 - Array analysis
2 - Matrix processing
3 - Bubble sort array
0 - Exit
Choose task: 1
Enter input file for array: array_in.txt
Task 1 completed. Check array_out.txt
```

```
===== MENU =====
1 - Array analysis
2 - Matrix processing
3 - Bubble sort array
0 - Exit
Choose task: 1
Enter input file for array: array_in-1.txt
Task 1 completed. Check array_out.txt
```

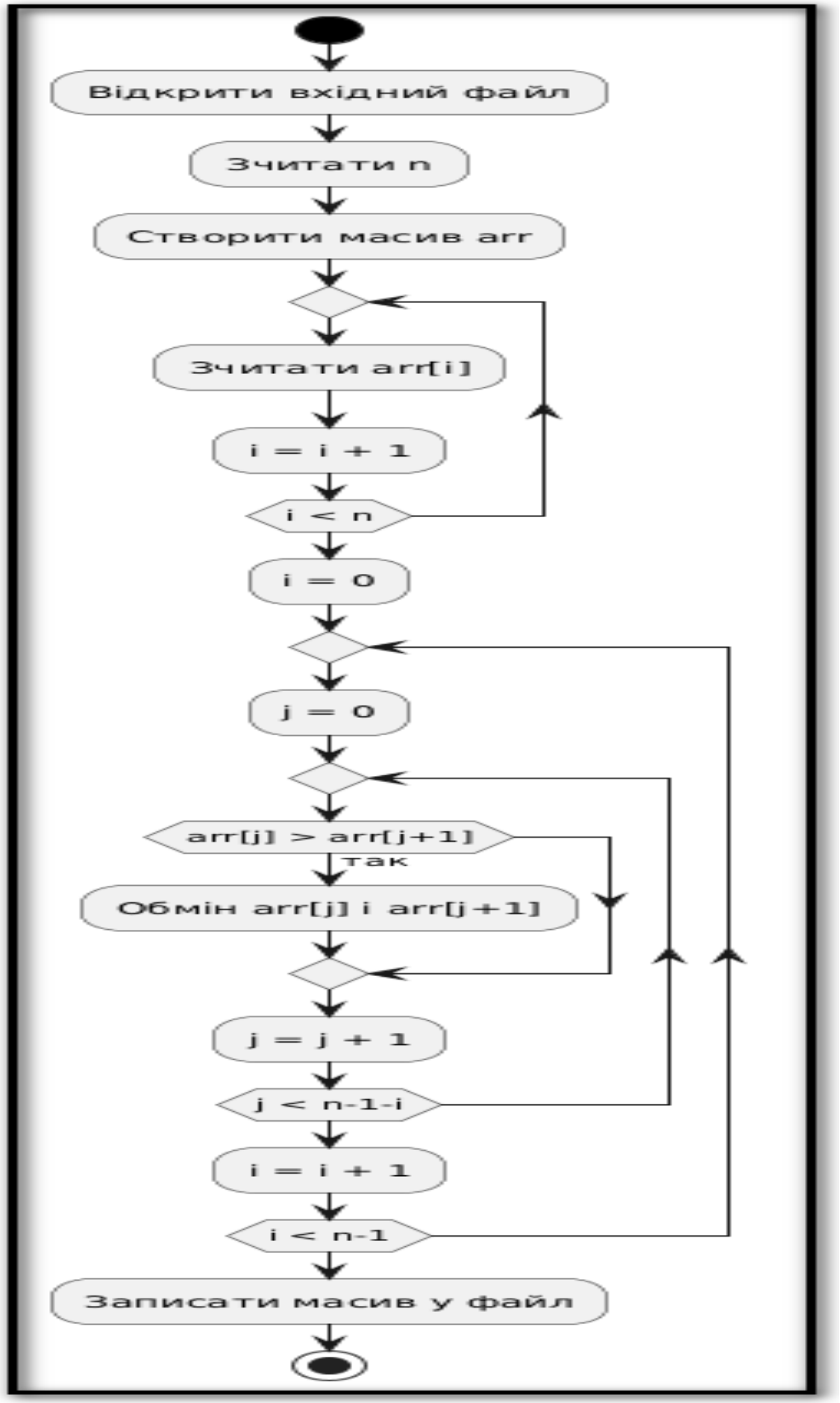
```
===== MENU =====
1 - Array analysis
2 - Matrix processing
3 - Bubble sort array
0 - Exit
Choose task: 2
Enter input file for matrix: matrix_in.txt
Task 2 completed. Check matrix_out.txt
```

```
===== MENU =====
1 - Array analysis
2 - Matrix processing
3 - Bubble sort array
0 - Exit
Choose task: 2
Enter input file for matrix: matrix_in-1.txt
Task 2 completed. Check matrix_out.txt
```

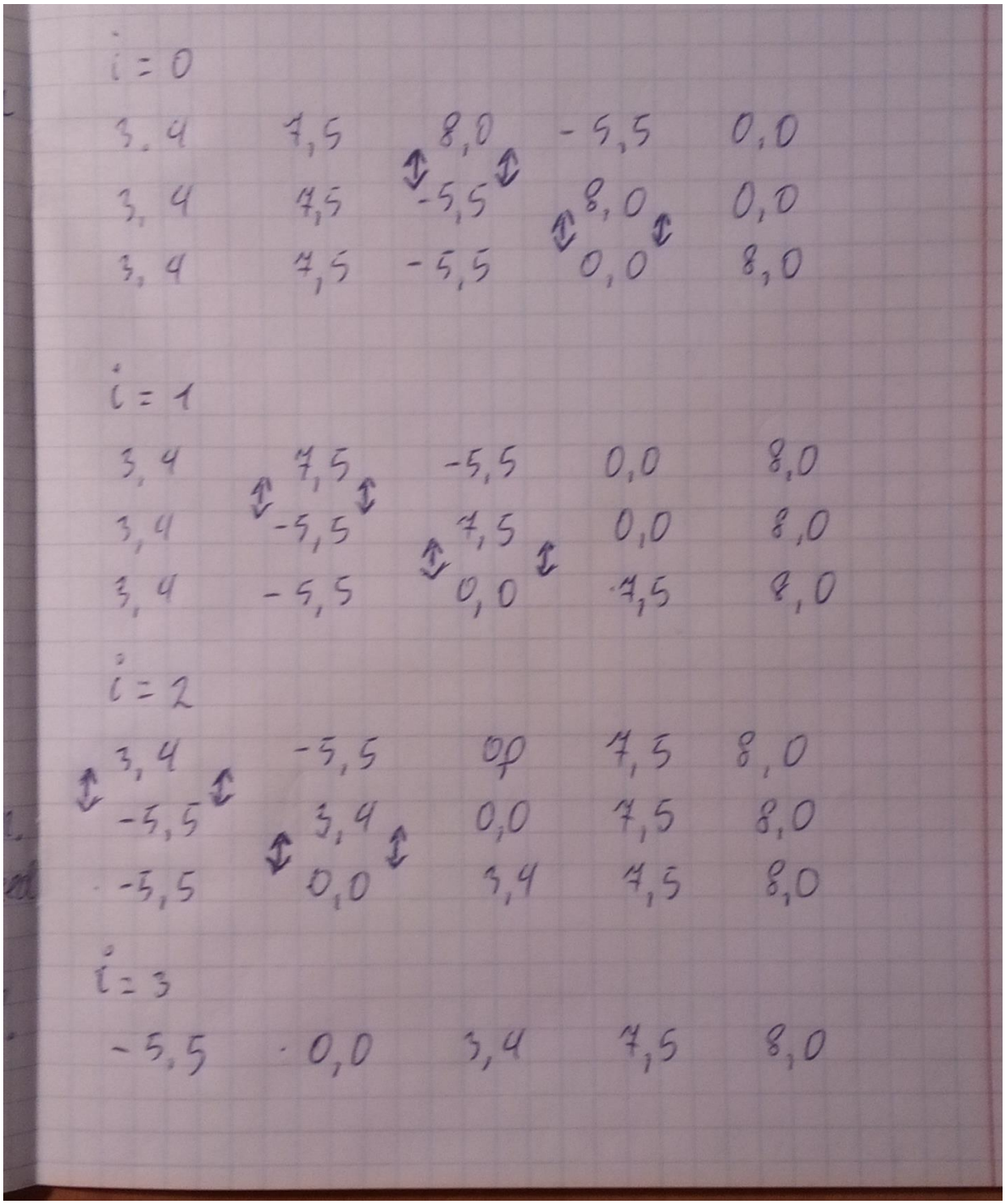
```
===== MENU =====
1 - Array analysis
2 - Matrix processing
3 - Bubble sort array
0 - Exit
Choose task: 3
Enter input file for array: array_out.txt
Task 3 completed. Check array_out.txt
```

ДОДАТОК В

Відкрити вхідний файл
Зчитати n
Створити масив arr
Зчитати arr[i]
i = i + 1
i < n


Відкрити вхідний файл
Зчитати n
Створити масив arr
Зчитати arr[i]
i = i + 1
i < n
i = 0
j = 0
arr[j] > arr[j+1]
так
Обмін arr[j] і arr[j+1]
j = j + 1
j < n-1-i
i = i + 1
i < n-1
Записати масив у файл

ДОДАТОК Г



i = 0

3,4 7,5 8,0 -5,5 0,0

3,4 7,5 -5,5 8,0 0,0

3,4 7,5 -5,5 0,0 8,0

i = 1

3,4 7,5 -5,5 0,0 8,0

3,4 -5,5 7,5 0,0 8,0

3,4 -5,5 0,0 7,5 8,0

i = 2

3,4 -5,5 0,0 7,5 8,0

-5,5 3,4 0,0 7,5 8,0

-5,5 0,0 3,4 7,5 8,0

i = 3

-5,5 0,0 3,4 7,5 8,0

У процесі сортування стрілками показані елементи, які порівнюються та міняються місцями. На кожному проході найбільший елемент переміщується в кінець масиву. Після кількох ітерацій масив стає повністю відсортованим.

Діалог з ШІ для самоаналізу

<u>Тепер ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4ма варіантами відповіді і три відкритих питання за кодом і теоретичними відомостями у файлах лекції.</u>
<u>+звіт</u>
<u>+текст лекції</u>

1. Тестові питання

1. Яка структура використовується для динамічного масиву у коді?
A) int arr[100]
B) vector<double>
C) list<int>
D) array<int, 10>

Правильна B)

2. Що робить функція swap(arr[j], arr[j+1])?
A) Копіює значення
B) Видаляє елемент
C) Міняє елементи місцями
D) Сортує масив

Правильна C)

3. Яка складність алгоритму бульбашкового сортування?
A) O(n)
B) O(log n)
C) $O(n^2)$
D) O(n log n)

Правильна C)

4. Для чого використовується ifstream?
A) Запис у файл
B) Зчитування з файлу
C) Видалення файлу
D) Робота з пам’яттю

Правильна B)

5. Що відбудеться, якщо файл не відкриється?
A) Програма завершиться автоматично
B) Дані будуть нульові
C) Може виникнути помилка читання
D) Масив автоматично заповниться

Правильна C)

2. Відкриті питання:

1. Поясни принцип роботи сортування бульбашкою.

Сортування бульбашкою працює так: ми кілька разів проходимо по масиву і порівнюємо сусідні елементи. Якщо вони стоять не в правильному порядку — міняємо їх місцями. У результаті після кожного проходу найбільші елементи поступово “переміщуються” в кінець масиву.

2. Чому використовується вкладений цикл у сортуванні?

Тому що одного проходу недостатньо, щоб відсортувати весь масив. Зовнішній цикл відповідає за кількість таких проходів, а внутрішній — безпосередньо порівнює елементи і виконує обмін.

3. Які помилки можуть виникнути при зчитуванні файлу?

Може бути таке, що файл не відкрився (наприклад, неправильне ім’я). Також можуть бути неправильні дані у файлі або їх менше, ніж очікується. У такому випадку програма може працювати неправильно або взагалі завершитись з помилкою.

4. Чим vector кращий за звичайний масив?

vector зручніший у використанні, бо не потрібно вручну слідкувати за пам’яттю. Він може змінювати розмір під час роботи програми і має готові функції. Звичайний масив більш обмежений і менш гнучкий.

5. Як можна оптимізувати бульбашкове сортування?
Можна зробити перевірку: якщо під час проходу не було жодного обміну, значить масив уже відсортований і можна зупинити алгоритм раніше. Це дозволяє зекономити час.

Оцінка відповідей на тестові запитання

Ці запитання перевіряють базові теоретичні знання. Відповіді на них були швидкими та точними, що може свідчити про використання допоміжних засобів, таких як ШІ. Тому, згідно з умовою, оцінка була зменшена на 60%.

Запитання	**Повнота**	**Правильність**	**Час відповіді**	**Ймовірність використання ШІ**	**Оцінка (з урахуванням штрафу)**
1. vector<double>	100%	100%	Миттєво	Висока	4/10
2. swap(arr[j], arr[j+1])	100%	100%	Миттєво	Висока	4/10
3. Складність $O(n^2)$	100%	100%	Миттєво	Висока	4/10
4. ifstream	100%	100%	Миттєво	Висока	4/10
5. Помилка відкриття файлу	100%	100%	Миттєво	Висока	4/10

Оцінка відповідей на відкриті запитання

Ці відповіді продемонстрували розуміння теми, логіки роботи алгоритму та особливостей реалізації коду. Відповіді сформульовані у більш простій та наближеній до розмовної формі, що знижує ймовірність використання ШІ, тому штраф не застосовувався.

Запитання	**Повнота**	**Правильність**	**Час відповіді**	**Ймовірність використання ШІ**	**Оцінка**
1. Принцип bubble sort	Повна	100%	Швидко	Низька	9/10
2. Вкладені цикли	Повна	100%	Швидко	Низька	9/10
3. Помилки зчитування файлу	Достатня	100%	Швидко	Низька	8/10
4. Переваги vector	Повна	100%	Швидко	Низька	9/10
5. Оптимізація сортування	Достатня	100%	Швидко	Низька	8/10

Середня оцінка:
(4 + 4 + 4 + 4 + 4 + 9 + 9 + 8 + 9 + 8) / 10 = 6.3 / 10